

EX.NO.8 INTER-THREAD COMMUNICATION

Program:

```
class Railway {
    private int availableSeats = 1; // initially 1 seat available

    public synchronized void bookTicket() {
        System.out.println("Trying to book ticket...");
        while (availableSeats == 0) {
            try {
                System.out.println("No seats available. Waiting for cancellation...");
                wait(); // wait until a seat is available
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
        availableSeats--;
        System.out.println("Ticket booked successfully!");
    }

    public synchronized void cancelTicket() {
        System.out.println("Cancelling ticket...");
        availableSeats++;
        System.out.println("Ticket cancelled. Seat is now available.");
        notify(); // notify waiting booking thread
    }
}

class BookingThread extends Thread {
    Railway railway;

    BookingThread(Railway railway) {
        this.railway = railway;
    }

    public void run() {
        railway.bookTicket();
    }
}

class CancellationThread extends Thread {
    Railway railway;

    CancellationThread(Railway railway) {
        this.railway = railway;
    }

    public void run() {
        railway.cancelTicket();
    }
}

public class RailwaySystem {
    public static void main(String[] args) throws InterruptedException {
        Railway railway = new Railway();
        BookingThread booking1 = new BookingThread(railway);
        BookingThread booking2 = new BookingThread(railway);
        CancellationThread cancel = new CancellationThread(railway);
        booking1.start();
        booking2.start();
        // Give time for booking threads to attempt
        Thread.sleep(1000);
        cancel.start();
    }
}
```

Output:

Trying to book ticket...
Ticket booked successfully!
Trying to book ticket...
No seats available. Waiting for cancellation...
Cancelling ticket...
Ticket cancelled. Seat is now available.
Ticket booked successfully!